

Лабораторная работа №7

Компьютерный практикум по статистическому анализу данных

Амуничников Антон Игоревич

2025-12-04

1. Информация
2. Вводная часть
3. Выполнение лабораторной работы
4. Самостоятельная работа

1. Информация

1.1 Докладчик

- Амуничников Антон Игоревич
- Группа: НПИбд-01-22
- Российский университет дружбы народов им. П. Лумумбы
- 1132227133@pfur.ru

2. Вводная часть

2.1 Цель работы

- Основной целью работы является освоение специализированных пакетов Julia для обработки данных.

2.2 Задание

1. Используя JupyterLab, повторите примеры. При этом дополните графики обозначениями осей координат, легендой с названиями траекторий, названиями графиков и т.п.
2. Выполните задания для самостоятельной работы.

3. Выполнение лабораторной работы

3.1 Выполнение примеров

```
[98]_ # Считывание данных и их запись в структуру:
P = CSV.File("Users/nasei/Downloads/programminglanguages.csv") |> DataFrame
first(P, 5)

[98]_ 5x2 DataFrame
      Row  year  language
      int64 string81
1      1  1951 Regional Assembly Language
2      2  1952 Autocode
3      3  1954 IPL
4      4  1955 FLOW-MATIC
5      5  1957 FORTRAN

[4]: # Функция определения по названию языка программирования года его создания:
function language_created_year(P, language::String)
    loc = findfirst(P[:,2].==language)
    return P[loc,1]
end

# Пример вызова функции и определение даты создания языка Python:
language_created_year(P,"Python")

# Пример вызова функции и определение даты создания языка Julia:
language_created_year(P,"Julia")

[4]: 2012

[5]: # Функция определения по названию языка программирования года его создания (без учёта регистра):
function language_created_year_v2(P, language::String)
    loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))
    return P[loc,1]
end

# Пример вызова функции и определение даты создания языка Julia:
language_created_year_v2(P,"Julia")

[5]: 2012

[7]: # Построение считывание данных с указанием разделителя:
Tx = readcsv("Users/nasei/Downloads/programminglanguages.csv", ",")

[7]: 74x2 Matrix{Any}:
"year"  "language"
1951    "Regional Assembly Language"
1952    "Autocode"
1954    "IPL"
1955    "FLOW-MATIC"
1957    "FORTRAN"
1957    "COMTRAN"
1958    "LISP"
1958    "ALGOL 58"
1959    "FACT"
1959    "COBOL"
1959    "RPG"
1962    "APL"
      :
2003    "Scala"
```

Рисунок 1: Считывание данных

3.2 Выполнение примеров

```
Запись данных в файл

[8]: # Запись данных в CSV-файл:
CSV.write("/Users/nasmi/Downloads/programming_languages_data2.csv", P)

[8]: "/Users/nasmi/Downloads/programming_languages_data2.csv"

[9]: # Пример записи данных в текстовый файл с разделителем ',':
writeln("/Users/nasmi/Downloads/programming_languages_data.txt", Tx, ',')

# Пример записи данных в текстовый файл с разделителем '-':
writeln("/Users/nasmi/Downloads/programming_languages_data2.txt", Tx, '-')

[10.. # Построчное считывание данных с указанием разделителя:
P_new_delim = readldl("/Users/nasmi/Downloads/programming_languages_data2.txt", '-')

[10.. 74x2 Matrix{Any}:
      "year"  "language"
1951      "Regional Assembly Language"
1952      "Autocode"
1954      "IPL"
1955      "FLOW-MATIC"
1957      "FORTRAN"
1957      "COMTRAN"
1958      "LISP"
1958      "ALGOL 58"
1959      "FACT"
1959      "COBOL"
1959      "RPG"
1962      "APL"
      :
2003      "Scala"
2005      "F#"
2006      "PowerShell"
2007      "Clojure"
2009      "Go"
2010      "Rust"
2011      "Dart"
2011      "Kotlin"
2011      "Red"
2011      "Elixir"
2012      "Julia"
2014      "Swift"
```

Рисунок 2: Запись данных в файл

3.3 Выполнение примеров

Словари

```
[11... # Инициализация словаря:
dict = Dict{Integer,Vector{String}}{()}

[11... Dict{Integer, Vector{String}}{()}

[12... # Инициализация словаря:
dict2 = Dict{Any, Any}{}

[12... Dict{Any, Any}{}

[13... # Заполнение словаря данными:
for i = 1:size(P,1)
    year, lang = P[i,:]

    if year in keys(dict)
        dict[year] = push!(dict[year], lang)
    else
        dict[year] = [lang]
    end
end

[14... dict[2003]

[14... 2-element Vector{String}:
 "Groovy"
 "Scala"
```

Рисунок 3: Словари

3.4 Выполнение примеров

DataFrames

```
[99.. # Подгружаем пакет DataFrames:
using DataFrames

# Задаём переменную со структурой DataFrame:
df = DataFrame{year = P{1,1}, language = P{1,2}}
first(df, 5)
```

[99.. 5x2 DataFrame

Row	year	language
1	1951	Regional Assembly Language
2	1952	Autocode
3	1954	IPL
4	1955	FLOW-MATIC
5	1957	FORTRAN

```
[16.. # Вывод всех значений столбца year:
df[!,:year]
```

[16.. 73-element Vector{Int64}:
1951
1952
1954
1955
1957
1957
1958
1958
1958
1959
1959
1959
1962
1962
:
2003
2005
2006
2007
2009
2010
2011
2011
2011
2012
2014

```
[17.. # Получение статистических сведений о фрейме:
describe(df)
```

[17.. 2x7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	year	1962.99	1951	1986.0	2014	0	Int64
2	language		ALGOL 58		dBase III	0	String31

Рисунок 4: DataFrames

3.5 Выполнение примеров

RDatasets

```
[18... # Подгружаем пакет RDatasets:
using RDatasets

# Задаём структуру данных в виде набора данных:
iris = dataset("datasets", "iris")
first(iris, 5)
```

[18... 5x5 DataFrame

Row	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa

```
[19... # Определения типа переменной:
typeof(iris)
```

[19... DataFrame

```
[20... describe(iris)
```

[20... 5x7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	SepalLength	5.84333	4.3	5.8	7.9	0	Float64
2	SepalWidth	3.05733	2.0	3.0	4.4	0	Float64
3	PetalLength	3.758	1.0	4.35	6.9	0	Float64
4	PetalWidth	1.19933	0.1	1.3	2.5	0	Float64
5	Species		setosa		virginica	0	CategoricalValue{String, UInt8}

Рисунок 5: RDatasets

3.6 Выполнение примеров

```
Работа с переменными отсутствующего типа (Missing Values)

[21] # Отсутствующий тип:
# = missing
typeof(a)

[22] Missing

[23] # Пример операции с переменной отсутствующего типа:
a+1

[22] missing

[23] # Определение перечня продуктов:
foods = ["apple", "cucumber", "tomato", "banana"]

# Определение калорий:
calories = [missing, 47, 22, 185]

[23] 4-element Vector{Union{Missing, Int64}}:
 missing
    47
    22
   185

[24] # Определение типа переменной:
typeof(calories)

[24] Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})

[25] # Подключаем пакет Statistics:
using Statistics
# Определение среднего значения:
mean(calories)

[25] missing

[26] # Определение среднего значения без значений с отсутствующим типом:
mean(skipmissing(calories))

[26] 58.8

[27] # Задание сведений о ценах:
prices = [0.85, 1.6, 0.8, 0.6]

# Формирование данных о калориях:
dataframe_calories = DataFrame(item=foods, calories=calories)

# Формирование данных о ценах:
dataframe_prices = DataFrame(item=foods, price=prices)

# Объединение данных о калориях и ценах:
DF = InnerJoin(dataframe_calories, dataframe_prices, on=:item)

[27] 4x3 DataFrame
 Row  item    calories price
   String  Int64?  Float64
1  apple    missing  0.85
2  cucumber    47     1.6
3  tomato     22     0.8
4  banana    missing  0.6
```

Рисунок 6: Работа с переменными отсутствующего типа (MissingValues)

3.7 Выполнение примеров

```
[31.. # Загрузка изображения:
X1 = load("/Users/nasmi/Downloads/images.png")

Warning: Output swatches are reduced due to the large size (180×280).
Load the ImageShow package for large images.
@ Colors ~/.julia/packages/Colors/VFEJ1/src/display.jl:159

[31..
```

The Julia logo is displayed, featuring the word "julia" in a bold, black, sans-serif font. Above the letters "i", "l", and "a" are four colored circles: a blue circle above the "i", a red circle above the "l", a green circle above the first "a", and a purple circle above the second "a".

```


[32.. # Определение типа и размера данных:
@show typeof(X1);
@show size(X1);

typeof(X1) = IndirectArrays.IndirectArray{ColorTypes.RGB{FixedPointNumbers.N0f8}, 2, UInt8, Matrix{UInt8}, OffsetArrays.OffsetVector{ColorTypes.RGB{FixedPointNumbers.N0f8}}, Vector{ColorTypes.RGB{FixedPointNumbers.N0f8}}}}}
size(X1) = (180, 280)
```

Рисунок 7: FileIO

3.8 Выполнение примеров

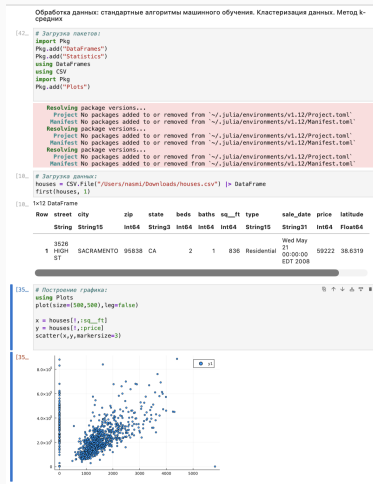


Рисунок 8: Кластеризация данных. Метод k-средних

3.9 Выполнение примеров

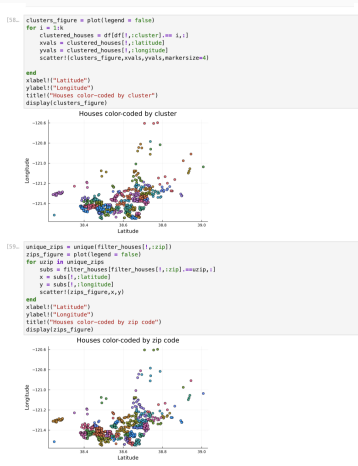


Рисунок 9: Кластеризация данных. Метод k-средних

3.10 Выполнение примеров

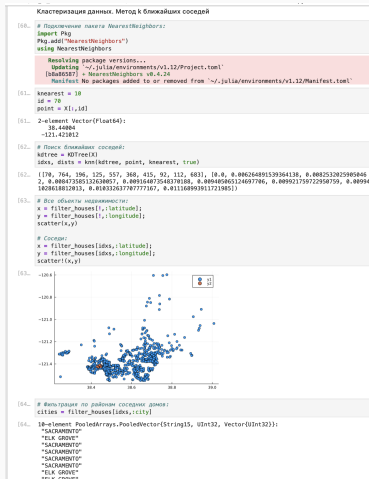


Рисунок 10: Кластеризация данных. Метод к ближайших соседей

3.11 Выполнение примеров

```
Обработка данных. Метод главных компонент

[66.] # Введи с указанием таблицы и цены недвижимости:
# F = filter_houses([lsq__ft, sprice])

# Конвертация данных в массив:
# F = convert(Array{Float64,2},F)'

F = filter_houses([, lsq__ft, sprice])
F = Matrix{Float64}{F}'

[66.] 2x814 adjoint{Matrix{Float64}} with eltype Float64:
 836.0  1167.0  796.0  852.0  797.0  1216.0  1605.0  1362.0
59222.0  68212.0  68808.0  65307.0  81900.0  235000.0  235300.0  235730.0

[67.] # Выводим пакет MultivariateStats:
import Plots
Plots.add!("MultivariateStats")
using MultivariateStats

Resolving package versions...
Installed Arpack_jll v3.5.1+1
Installed MultivariateStats v0.18.3
Installed Arpack v0.5.4
Installing 1 artifacts
Installed artifact Arpack 203.4 KiB
Updating "/.julia/environments/v1.12/Project.toml"
[67286f6a] + MultivariateStats v0.18.3
Updating "/.julia/environments/v1.12/Manifest.toml"
[7d9fca2a] + Arpack v0.5.4
[67286f6a] + MultivariateStats v0.18.3
[68021587] + Arpack_jll v3.5.1+1
Info Packages marked with ~ have new versions available but compatibility constraints restrict them from upgrading. To see why use "status --outdated -m"
Precompiling packages...
 473.8 ms / Arpack_jll
1204.7 ms / Arpack
1500.5 ms / MultivariateStats
3 dependencies successfully precompiled in 4 seconds. 500 already precompiled.

[68.] # Приведение типов данных к распределению для PCA:
M = fit{PCA, F}

[68.] PCA{indim = 2, outdim = 1, principalratio = 0.9999840784692897}

Pattern matrix (unstandardized loadings):

      PCI
1  460.52
2  1.19826e5

Importance of components:

      PCI
SS Loadings (Eigenvalues) 1.43584e10
Variance explained 0.999984
Cumulative variance 0.999984
Proportion explained 1.0
Cumulative proportion 1.0

[68.] # Выводим значений главных компонент в отдельную переменную:
Xr = reconstruct(M, y)

# Построение графика с выделением главных компонент?
.....
```

Рисунок 11: Обработка данных. Метод главных компонент

3.12 Выполнение примеров

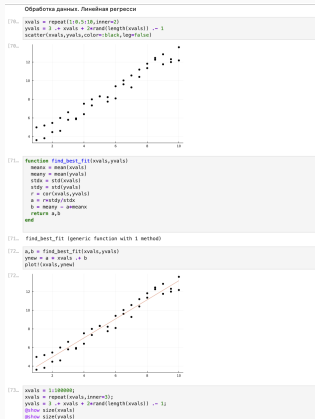


Рисунок 12: Обработка данных. Линейная регрессия

3.13 Выполнение примеров

```
[73... xvals = 1:100000;  
xvals = repeat(xvals,inner=3);  
yvals = 3 .* xvals + 2*rand(length(xvals)) .* 1;  
@show size(xvals)  
@show size(yvals)  
  
size(xvals) = (300000,  
size(yvals) = (300000,  
[73... (300000,  
  
[74... @time a,b = find_best_fit(xvals,yvals)  
0.016666 seconds (33.27 k allocations: 1.666 MiB, 95.81% compilation time)  
[74... (1.0000000281329204, 2.9982584778408636)
```

Рисунок 13: Обработка данных. Линейная регрессия

4. Самостоятельная работа

4.1 Задание 1: Кластеризация

```
107. using Rdatasets, Clustering, Plots, DataFrames, Statistics

# Загрузка данных Iris
iris = dataset("datasets", "Iris")
println("Первые 5 строк данных Iris:")
println(first(iris, 5)) # Искрашено: iris вместо Iris

# Подготовка данных для кластеризации (используем только числовые признаки)
X_iris = Matrix{Float64, 2}(iris[1:4,:]) # Искрашено: 1:4 вместо 1:4, правильные кавычки
println("Размерность данных: ", size(X_iris))

# Кластеризация методом k-средних
k = 3 # Известно, что в Iris 3 класса
result = kmeans(X_iris, k)

# Добавим метки кластеров в DataFrame
iris_with_clusters = copy(iris)
iris_with_clusters[:, :Cluster] = result.assigned # Искрашено: правильный синтаксис индексации

# Визуализация кластеров
p1 = scatter(iris_with_clusters[:, :SepalLength], iris_with_clusters[:, :SepalWidth], # Искрашено:
    group=:iris_with_clusters[:, :Cluster], markersize=:alpha*0.7,
    title="Кластеризация Iris (Sepal)", xlabel="Ширина чашелистика", ylabel="Высота чашелистика")
p2 = scatter(iris_with_clusters[:, :PetalLength], iris_with_clusters[:, :PetalWidth], # Искрашено:
    group=:iris_with_clusters[:, :Cluster], markersize=:alpha*0.7,
    title="Кластеризация Iris (Petal)", xlabel="Ширина лепестка", ylabel="Высота лепестка")

plot(p1, p2, layout=(2,2), size=(1000, 400), legend=:topleft) # Искрашено: :topleft вместо :top

# Сравнение с реальными классами
println("Сравнение кластеров с реальными классами:")
# c1 = combine(groupby(iris_with_clusters, [:Species, :Cluster]), rows => :Count)
println(c1) # Искрашено: c1 вместо c

# Статистика по кластерам
println("Центры кластеров:")
display(result.centers')

# Дополнительный анализ - точность кластеризации
println("Точность кластеризации:")
# Для каждого кластера находим наиболее частый вид
cluster_species = combine(groupby(iris_with_clusters, :Cluster)) do df
    dominant_species = first(sort(combine(groupby(df, :Species), rows => :Count), by=:count, rev=true))
    DataFrame{DominantSpecies = dominant_species, Count = rowcount}
end

println("Доминирующие виды в кластерах:")
display(cluster_species)

# Вычислим accuracy
correct = 0
total = nrow(iris_with_clusters)
for i in 1:nrow(iris_with_clusters)
    cluster_sum = iris_with_clusters[i, :Cluster]
    actual_species = iris_with_clusters[i, :Species]
    predicted_species = cluster_species[cluster_sum, :DominantSpecies]
    if actual_species == predicted_species
        correct += 1
    end
end
end
```

Рисунок 14: Кластеризация

4.2 Задание 1: Кластеризация

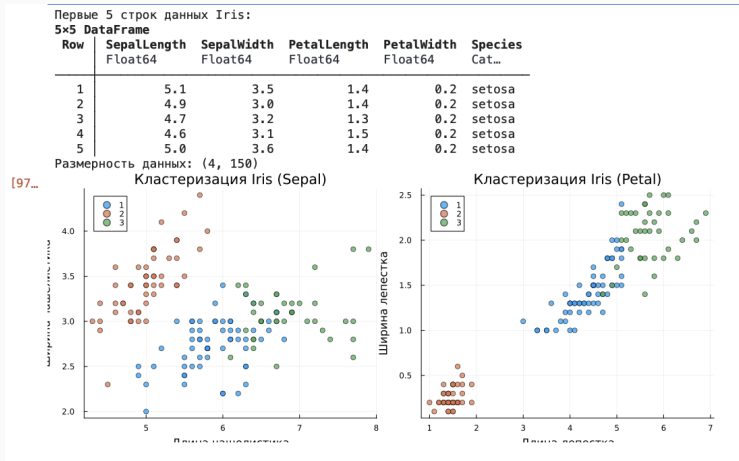


Рисунок 15: Кластеризация

4.3 Задание 2: Регрессия

Пусть регрессионная зависимость является линейной. Матрица наблюдений факторов X имеет размерность $N \times 3$, массив результатов в $N \times 1$, регрессионная зависимость является линейной. Найдем МНК-оценку для линейной модели.

4.4 Задание 2: Регрессия

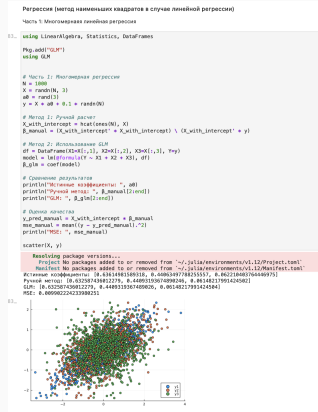


Рисунок 16: Регрессия

4.5 Задание 2: Регрессия

Найдем линию регрессии, используя данные (X, y) . Построим график (X, y) , используя точечный график. Добавим линию регрессии, используя `abline`!. Добавим заголовок «График регрессии» и подпишем оси x и y .

4.6 Задание 2: Регрессия

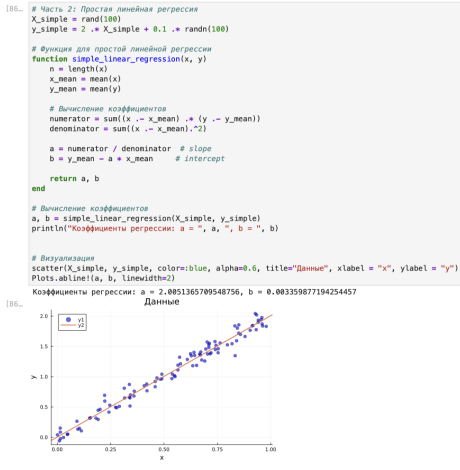


Рисунок 17: Регрессия

4.7 Задание 3: Модель ценообразования биномиальных опционов

Построим траекторию возможных цен на акции:

- S – начальная цена акции;
- T – длина биномиального дерева в годах;
- n – количество периодов;
- $h = Tn$ – длина одного периода;
- σ – волатильность акции;
- r – годовая процентная ставка;
- $u = \exp(rh + \sigma\sqrt{h})$;
- $d = \exp(rh - \sigma\sqrt{h})$;
- $p^* = \frac{\exp(rh) - d}{u - d}$;

4.8 Задание 3: Модель ценообразования биномиальных опционов

Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуем построить траекторию курса акций.

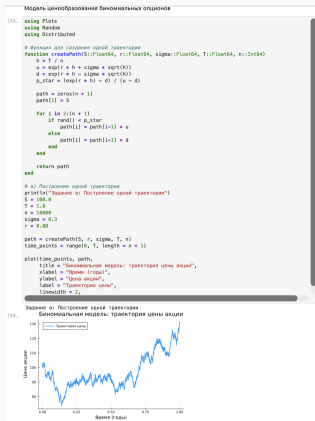


Рисунок 18: Модель ценообразования биномиальных опционов

4.9 Задание 3: Модель ценообразования биномиальных опционов

Создадим функцию `createPath` (`S :: Float64`, `r :: Float64`, `sigma :: Float64`, `T :: Float64`, `n :: Int64`), которая создает траекторию цены акции с учетом начальных параметров. Используем `createPath`, чтобы создать 10 разных траекторий и построим их все на одном графике.

4.10 Задание 3: Модель ценообразования биномиальных опционов

```
[92... # b) Построение 10 траекторий на одном графике
println("\nЗадание b: Построение 10 траекторий")
time_points = range(0, T, length = n + 1)
plt = plot(title = "10 траекторий цен акций",
           xlabel = "Время (годы)",
           ylabel = "Цена акции",
           legend = false)

for i in 1:10
    path = createPath(S, r, sigma, T, n)
    plot!(plt, time_points, path, linewidth = 1, alpha = 0.7)
end

display(plt)
```

Задание b: Построение 10 траекторий
10 траекторий цен акций

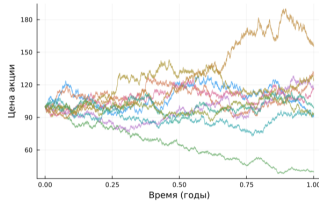


Рисунок 19: Модель ценообразования биномиальных опционов

4.11 Задание 3: Модель ценообразования биномиальных опционов

Распараллелим генерацию траектории. Можем использовать `Threads.@threads`, `pmap` и `@parallel`.

```
[93... # c) Распараллеленная версия с использованием Threads
println("\nЗадание c: Тестирование параллельных методов")

# Проверим доступное количество потоков
println("Доступное количество потоков: ", Threads.nthreads())

function createPaths_threads(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64, num_p
    paths = Vector{Vector{Float64}}{undef, num_paths}

    Threads.@threads for i in 1:num_paths
        paths[i] = createPath(S, r, sigma, T, n)
    end

    return paths
end

# Тестируем на меньшем количестве для скорости
n_test = 1000
num_paths_test = 50

println("Последовательная версия (", num_paths_test, " траекторий):")
@time paths_seq = [createPath(S, r, sigma, T, n_test) for _ in 1:num_paths_test]

println("Параллельная версия с Threads.@threads:")
```

Задание c: Тестирование параллельных методов
Доступное количество потоков: 1
Последовательная версия (50 траекторий):
0.019547 seconds (36.86 k allocations: 2.178 MiB, 97.20% compilation time)
Параллельная версия с Threads.@threads:
0.041200 seconds (80.83 k allocations: 4.507 MiB, 98.90% compilation time)

```
[93... 50-element Vector{Vector{Float64}}:
 [100.0, 100.96127414097793, 101.93178876169699, 102.91163268849944, 103.90089568159217, 102.92809
 986706874, 103.91752107489081, 104.91645332929082, 103.93414920847799, 102.96104212950706, ..., 162.
 7746199486427, 164.34219964436187, 165.92197871225665, 167.51694378781647, 165.94852835276518, 16
 4.39479756346546, 162.85561393158685, 164.42118203544125, 162.88167291447814, 161.35665625580734]
 [100.0, 99.06372728657354, 100.01680120806827, 100.97742923721371, 100.03200512854616, 99.0954327
 5198912, 100.04801152184339, 101.0097471851655, 100.06402048436973, 101.02591003770884, ..., 105.213
 29672560354, 104.22821333746529, 105.23813219988165, 104.24489118579156, 103.26887471447787, 102.3
 0199641506366, 101.34417074138144, 100.39531292408231, 101.36838710589462, 102.33473829633871]
 [100.0, 99.06372728657354, 100.01680120806827, 100.97742923721371, 101.94889915269533, 102.928099
 86706874, 103.91752107489081, 102.94456968059741, 103.934149208478, 104.93324138046452, ..., 134.645
 7619959251, 133.38511046657212, 132.13626207349084, 133.40645379166722, 132.15748556686595, 130.92
 404147077187, 130.40439411700146, 130.47000894247876, 131.77798787605716, 130.48407708444471]
```

Рисунок 20: Распараллелим генерацию траектории

4.12 Выводы

- В результате выполнения данной лабораторной работы я освоил специализированные пакеты Julia для обработки данных.